1

Pengantar Python

Python adalah bahasa pemrograman tingkat tinggi yang diciptakan oleh *Guido van Rossum,* seorang programmer asal Belanda. Python pertama kali dirilis pada tahun 1991, dan dikembangkan sebagai proyek sampingan selama libur Natal pada akhir tahun 1980-an. Nama "Python" sendiri diambil dari acara komedi Inggris, *Monty Python's Flying Circus*, yang merupakan acara favorit Guido van Rossum pada saat itu.

Berikut adalah sejarah singkat perkembangan Python dari awal hingga menjadi bahasa pemrograman populer saat ini:

1. **1980-an (Awal Pengembangan)**

   Guido van Rossum mulai mengembangkan Python pada akhir tahun 1980-an sebagai penerus dari bahasa ABC, sebuah bahasa yang dirancang untuk pemula dan mudah digunakan. Namun, Python dikembangkan dengan kemampuan untuk menangani berbagai kebutuhan pemrograman tingkat lanjut, termasuk sistem operasi dan pengembangan aplikasi jaringan.

2. **Python 1.0 (1991)**

   Python versi pertama dirilis pada tahun 1991. Versi ini memperkenalkan fitur-fitur dasar Python, seperti pengecekan tipe dinamis, kelas, fungsi, serta tipe data seperti list, dictionary, dan string. Python 1.0 sudah mendukung pemrograman berorientasi objek, fungsional, dan terstruktur, sehingga menjadi bahasa pemrograman yang serbaguna.

3. **Python 2.0 (2000)**

   Python 2.0 dirilis pada tahun 2000 dan merupakan lompatan besar dalam pengembangan bahasa ini. Fitur penting yang diperkenalkan di Python 2.0 adalah *garbage collection*, sistem untuk mengelola memori secara otomatis, dan *list comprehension*, yang memudahkan dalam memproses elemen-elemen di dalam list. Python 2.x tetap populer hingga bertahun-tahun dan menjadi fondasi pengembangan perangkat lunak di banyak perusahaan.

4. **Python 3.0 (2008)**

   Python 3.0, dirilis pada tahun 2008, membawa perubahan besar yang menyebabkan ketidakcocokan dengan Python 2.x. Tujuan utama Python 3.0 adalah untuk menghilangkan kekurangan yang ada pada Python 2.x, membuat bahasa ini lebih konsisten dan mudah dipahami. Beberapa perubahan penting di Python 3 adalah penyederhanaan fungsi *print*, penggunaan Unicode sebagai standar untuk string, dan penggantian cara penanganan angka integer.

5. **Era Python 3.x dan Popularitas Python**

   Setelah beberapa tahun, Python 3.x semakin diterima oleh komunitas, dan banyak pustaka serta framework yang diperbarui untuk mendukung Python 3. Python mulai dikenal sebagai bahasa yang sangat serbaguna dan banyak digunakan di bidang data science, kecerdasan buatan (AI), pengembangan web, dan otomasi sistem.

6. **Python sebagai Bahasa Populer (2010-an - Sekarang)**

   Sejak pertengahan 2010-an, Python menjadi salah satu bahasa pemrograman yang paling populer di dunia. Python dikenal dengan sintaksisnya yang sederhana dan mudah dipahami, sehingga sering dipilih oleh pemula. Selain itu, Python memiliki komunitas yang aktif dan dukungan pustaka yang luas, seperti NumPy dan Pandas untuk analisis data, TensorFlow dan PyTorch untuk pembelajaran mesin, serta Django dan Flask untuk pengembangan web.

Python terus berkembang, dan hingga saat ini Python tetap menjadi pilihan utama dalam berbagai aplikasi, dari pengembangan web hingga data science dan machine learning.

Kelebihan dan kerungan python dibandingkan bahasa C/C++

Python dan C/C++ adalah bahasa pemrograman yang sering dibandingkan karena memiliki kelebihan dan kekurangan yang berbeda sesuai dengan kebutuhan penggunanya. Berikut adalah beberapa kelebihan dan kekurangan Python dibandingkan dengan C/C++:

Kelebihan Python dibandingkan C/C++

1. **Sintaksis yang Mudah dan Sederhana**
   - Python memiliki sintaksis yang lebih sederhana dan mirip dengan bahasa manusia, sehingga lebih mudah dipahami, dipelajari, dan ditulis oleh pemula

maupun profesional. Di Python, tidak perlu mendeklarasikan tipe data secara eksplisit, dan penanganan memori dilakukan otomatis dengan *garbage collector*.

- C/C++ memiliki sintaksis yang lebih kompleks dan kaku. Struktur bahasa ini mengharuskan penggunaan tanda kurung kurawal {} dan titik koma ;, serta deklarasi tipe data untuk variabel.

2. **Produktivitas yang Tinggi**
   - Karena sintaksis Python yang sederhana, pengembangan dan debugging di Python menjadi lebih cepat. Hal ini membuat Python sangat cocok untuk *rapid prototyping*, pengembangan aplikasi sederhana, dan data science.
   - C/C++ membutuhkan kode yang lebih panjang dan waktu pengembangan lebih lama untuk aplikasi yang sama.
3. **Pustaka dan Ekosistem yang Kaya**
   - Python memiliki pustaka yang sangat luas, terutama untuk data science, kecerdasan buatan, *web development*, dan otomatisasi. Pustaka seperti NumPy, Pandas, TensorFlow, dan Django menjadikan Python sebagai bahasa utama di banyak bidang.
   - C++ juga memiliki pustaka yang banyak, tetapi pustaka yang ada di Python lebih unggul di bidang-bidang tertentu, seperti analisis data dan AI.
4. **Manajemen Memori Otomatis**
   - Python memiliki sistem *garbage collection* otomatis, yang secara otomatis membebaskan memori yang tidak digunakan. Ini mengurangi risiko kesalahan manajemen memori.
   - C++ mengharuskan pengembang untuk mengelola memori secara manual dengan mengalokasikan dan menghapus memori, yang dapat meningkatkan risiko kebocoran memori dan kesalahan lain yang berhubungan dengan memori.
5. **Kompatibilitas dan Keterbacaan Kode**
   - Kode Python yang sederhana membuatnya lebih mudah untuk di-*debug*, diuji, dan dipelihara. Python adalah bahasa yang interpretatif, yang memungkinkan menjalankan kode secara langsung tanpa perlu dikompilasi terlebih dahulu.
   - C++ perlu dikompilasi menjadi *binary* sebelum dieksekusi, dan keterbacaan kode sering kali tidak sebaik Python.

Kekurangan Python dibandingkan C/C++

1. **Kecepatan Eksekusi yang Lebih Lambat**
   - Python adalah bahasa interpretatif yang relatif lambat dibandingkan C/C++, yang merupakan bahasa terkompilasi. Python bukan pilihan terbaik untuk aplikasi yang membutuhkan kinerja tinggi, seperti game yang sangat intensif grafis atau sistem operasi.
   - C/C++ dikompilasi menjadi *machine code*, sehingga berjalan jauh lebih cepat dan sangat efisien untuk aplikasi yang membutuhkan kinerja optimal.
2. **Kontrol Lebih Rendah Terhadap Sistem**
   - Python tidak memberikan akses langsung ke perangkat keras dan kontrol rendah terhadap sistem, seperti pemrosesan memori dan perangkat input-output. Hal ini menjadikannya kurang ideal untuk pengembangan *low-level* dan aplikasi yang memerlukan optimasi tinggi.
   - C/C++ memungkinkan pengembang untuk mengakses dan mengontrol perangkat keras secara langsung, sehingga lebih cocok untuk aplikasi sistem, perangkat lunak tertanam (*embedded software*), dan perangkat keras.
3. **Kurangnya Dukungan untuk *Multithreading* Sebenarnya**
   - Python memiliki Global Interpreter Lock (GIL), yang membatasi eksekusi *thread* secara paralel di lingkungan multi-core. Hal ini membuat Python kurang optimal dalam memanfaatkan beberapa CPU core untuk tugas *multithreading*.
   - C++ mendukung *true multithreading* tanpa pembatasan seperti GIL, sehingga lebih efisien untuk aplikasi yang membutuhkan *multithreading* tingkat tinggi.
4. **Ukuran File Eksekusi yang Lebih Besar**
   - Program Python membutuhkan interpreter atau pustaka eksternal untuk dieksekusi, yang dapat meningkatkan ukuran file eksekusi akhir.
   - C++ menghasilkan file eksekusi yang lebih kecil karena langsung dikompilasi menjadi *machine code* tanpa perlu interpreter eksternal.
5. **Kurang Ideal untuk *Real-Time* dan *Embedded Systems***
   - Python jarang digunakan dalam sistem *embedded* atau aplikasi waktu nyata (*real-time applications*) karena keterbatasan kinerjanya.

- C++ adalah pilihan yang lebih baik untuk aplikasi ini, karena memungkinkan pengembangan program yang cepat dan responsif.

Kesimpulan

Python cocok untuk pengembangan aplikasi yang memerlukan produktivitas tinggi, prototyping cepat, dan analisis data. Di sisi lain, C/C++ lebih cocok untuk aplikasi yang membutuhkan kinerja tinggi dan kontrol rendah terhadap perangkat keras, seperti aplikasi sistem dan *embedded systems*.

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

2

Install dan Konfigurasi Python

Berikut adalah langkah-langkah untuk menginstall dan mengkonfigurasi Python di sistem operasi Windows, macOS, atau Linux:

Langkah 1: Download Python

1. **Buka Situs Resmi Python**

   Kunjungi https://www.python.org/downloads/ untuk mendownload versi terbaru Python.

2. **Pilih Versi Python**

   Pilih versi Python yang sesuai dengan sistem operasi Anda (Windows, macOS, atau Linux) dan klik "Download".

Langkah 2: Install Python

Di Windows

1. **Jalankan Installer**

   Buka file yang sudah didownload, dan pastikan untuk mencentang opsi *"Add Python to PATH"* di halaman awal installer.

2. **Pilih Install Now**

   Pilih opsi "Install Now" atau "Customize Installation" jika Anda ingin mengatur lokasi instalasi secara manual.

3. **Verifikasi Instalasi**

   Setelah instalasi selesai, buka Command Prompt dan ketik:

   ```
   python –version
   ```

   Anda akan melihat versi Python yang terinstal jika berhasil.

Di macOS

1. **Installer untuk macOS**

   Setelah mendownload file installer (file .pkg), buka file tersebut dan ikuti instruksi instalasi.

2. **Verifikasi Instalasi**

Setelah instalasi selesai, buka Terminal dan ketik:

```
python3 --version
```

Versi Python akan ditampilkan jika instalasi berhasil.

Di Linux

1. **Paket Manajer (APT untuk Ubuntu/Debian)**

   Di sebagian besar distribusi Linux, Python sudah terpasang secara default. Untuk memastikan versi terbaru, Anda dapat memperbarui Python dengan perintah berikut:

   ```
   sudo apt update
   sudo apt install python3
   ```

2. **Verifikasi Instalasi**

   Buka Terminal dan ketik:

   ```
   python3 --version
   ```

   Versi Python akan ditampilkan jika instalasi berhasil.

Langkah 3: Konfigurasi Lingkungan Python (Opsional)

1. **Menambahkan Python ke PATH (Windows Manual)**

   Jika Anda tidak mencentang *Add Python to PATH* saat instalasi, Anda perlu menambahkannya secara manual. Masuk ke:

   - *Control Panel > System and Security > System > Advanced system settings*
   - Klik pada "Environment Variables", cari variabel "Path", dan tambahkan lokasi direktori Python (misal, C:\Python39).

2. **Install PIP (Jika Belum Ada)**

   PIP adalah package manager untuk Python yang memungkinkan Anda menginstall pustaka pihak ketiga. PIP biasanya sudah terinstal bersama Python, tapi jika belum, Anda bisa menginstallnya dengan:

   ```
   python -m ensurepip --upgrade
   ```

3. **Membuat Virtual Environment (Opsional)**

   Virtual Environment (venv) memungkinkan Anda mengisolasi pustaka untuk setiap proyek. Untuk membuat *virtual environment*:

```
python -m venv nama_environment
```

Untuk mengaktifkan *environment*:

- **Windows**: nama_environment\Scripts\activate
- **macOS/Linux**: source nama_environment/bin/activate

4. **Menginstall Pustaka (Contoh)**

   Anda dapat menginstall pustaka atau modul tambahan dengan PIP. Misalnya, untuk menginstall pustaka requests:

```
pip install requests
```

Langkah 4: Pengujian Python

1. **Buka Terminal atau Command Prompt**

   Ketik python atau python3, dan Anda akan masuk ke mode interaktif Python (REPL) di mana Anda dapat mengetik perintah Python langsung.

2. **Coba Program Sederhana**

   Buat file Python baru (hello.py) dengan teks berikut:

```
print("Hello, World!")
```

   Kemudian jalankan dengan:

```
python hello.py
```

Itulah cara menginstal dan mengkonfigurasi Python di berbagai sistem operasi.

3

Dasar-Dasar Python

Dasar-dasar Python meliputi sintaksis, tipe data, operasi dasar, struktur kontrol, dan fungsi-fungsi dasar. Python adalah bahasa yang mudah dipelajari karena memiliki sintaksis yang sederhana. Berikut ini adalah beberapa konsep dasar dalam Python:

1. **Menulis Program Python**
   - **Print Statement**

     print() digunakan untuk menampilkan output ke layar.

     ```
     print("Hello, World!")
     ```
   - **Komentar**

     Komentar dimulai dengan tanda # dan digunakan untuk memberi keterangan pada kode.

     ```
     # Ini adalah komentar
     print("Komentar tidak dieksekusi")
     ```
2. **Variabel dan Tipe Data**

   Python adalah bahasa yang dinamis, artinya Anda tidak perlu mendeklarasikan tipe data variabel. Berikut adalah beberapa tipe data dasar:

   A. **Tipe Data Dasar**

   ```
   x = 5          # Integer
   y = 3.14       # Float
   name = "Alice"   # String
   is_active = True # Boolean
   ```

   B. **Tipe Data Koleksi**
   - **List**: Tipe data terurut yang dapat diubah.

     ```
     fruits = ["apple", "banana", "cherry"]
     ```
   - **Tuple**: Tipe data terurut yang tidak dapat diubah.

     ```
     coordinates = (10, 20)
     ```

- **Set**: Koleksi tidak terurut yang tidak memiliki duplikasi.

```
unique_numbers = {1, 2, 3, 4}
```

C. **Dictionary**: Koleksi data pasangan kunci-nilai.

```
student = {"name": "Alice", "age": 20}
```

3. Operasi Dasar

A. Operasi Aritmatika

```
a = 10
b = 3
print(a + b)  # Penjumlahan
print(a - b)  # Pengurangan
print(a * b)  # Perkalian
print(a / b)  # Pembagian
print(a % b)  # Modulus
print(a ** b) # Pangkat
```

B. Operasi Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai, dan hasilnya adalah nilai boolean (True atau False).

```
print(a == b) # Sama dengan
print(a != b) # Tidak sama dengan
print(a > b)  # Lebih besar
print(a < b)  # Lebih kecil
```

C. Operasi Logika

Operator logika digunakan untuk operasi boolean.

```
print(True and False) # False
print(True or False)  # True
print(not True)       # False
```

4. Struktur Kontrol

A. Percabangan (if-elif-else)

Digunakan untuk membuat keputusan berdasarkan kondisi.

```
x = 10
if x > 5:
    print("x lebih besar dari 5")
elif x == 5:
    print("x sama dengan 5")
else:
    print("x lebih kecil dari 5")
```

B. Perulangan (for dan while)

- **for** digunakan untuk iterasi dalam koleksi atau rentang angka.

```
for i in range(5):
    print(i)
```

- **while** digunakan untuk melakukan perulangan berdasarkan kondisi.

```
i = 0
while i < 5:
    print(i)
    i += 1
```

5. Fungsi

Fungsi adalah blok kode yang dapat digunakan kembali. Fungsi dideklarasikan dengan kata kunci def.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")  # Output: Hello, Alice!
```

Fungsi juga dapat mengembalikan nilai menggunakan return.

```
def add(a, b):
    return a + b

result = add(3, 4)
```

```
print(result)  # Output: 7
```

6. **Input dari Pengguna**

   Python memungkinkan kita untuk mengambil input dari pengguna menggunakan input().

```
name = input("Masukkan nama Anda: ")
print("Hello, " + name)
```

7. **Handling Error (Exception)**

   Python memiliki sistem untuk menangani kesalahan atau *exception*. Misalnya, try-except digunakan untuk menangkap dan menangani error.

```
try:
    angka = int(input("Masukkan angka: "))
    hasil = 10 / angka
    print("Hasil:", hasil)
except ZeroDivisionError:
    print("Tidak bisa membagi dengan nol!")
except ValueError:
    print("Input harus berupa angka!")
```

8. **Mengimport Modul**

   Python memiliki pustaka modul yang sangat luas. Modul dapat diimpor menggunakan import.

```
import math
print(math.sqrt(16))  # Output: 4.0
```

9. **List Comprehension**

   List comprehension adalah cara singkat untuk membuat list berdasarkan kondisi atau ekspresi tertentu.

```
squares = [x**2 for x in range(10)]
print(squares)  # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

10. Membaca dan Menulis File

Python menyediakan API untuk membaca dan menulis file.

```
# Menulis ke file
with open("file.txt", "w") as file:
    file.write("Hello, World!")

# Membaca dari file
with open("file.txt", "r") as file:
    content = file.read()
    print(content)
```

Ini adalah dasar-dasar Python yang bisa membantu Anda memahami konsep utama dalam bahasa pemrograman ini. Selamat belajar dan mencoba!

TEKNIK ... RINGAN
SMK NEGERI SUKASARI

4

Operators

Python memiliki berbagai jenis operator yang bisa digunakan untuk melakukan operasi aritmatika, perbandingan, logika, dan lainnya. Berikut adalah beberapa kategori operator dalam Python beserta contoh penggunaannya:

1. Operator Aritmatika

Digunakan untuk melakukan operasi matematika dasar.

Operator	Deskripsi	Contoh	Hasil
+	Penjumlahan	5 + 3	8
-	Pengurangan	5 - 3	2
*	Perkalian	5 * 3	15
/	Pembagian	5 / 3	1.666
%	Modulus	5 % 3	2
**	Pangkat	5 ** 3	125
//	Pembagian Bulat	5 // 3	1

```
a = 10
b = 3
print(a + b)  # 13
print(a - b)  # 7
print(a * b)  # 30
print(a / b)  # 3.3333...
print(a % b)  # 1
print(a ** b) # 1000
print(a // b) # 3
```

2. Operator Perbandingan

Digunakan untuk membandingkan dua nilai dan menghasilkan nilai boolean (True atau False).

Operator	Deskripsi	Contoh	Hasil
==	Sama dengan	5 == 3	False
!=	Tidak sama dengan	5 ! == 3	True
>	Lebih besar	5 > 3	True
<	Lebih kecil	5 < 3	False
>=	Lebih besar atau sama dengan	5 >= 3	True
<=	Lebih kecil atau sama dengan	5 <= 3	False

```
a = 10
b = 5
print(a == b)  # False
print(a != b)  # True
print(a > b)   # True
print(a < b)   # False
print(a >= b)  # True
print(a <= b)  # False
```

3. Operator Logika

Digunakan untuk operasi logika dan menghasilkan nilai boolean.

Operator	Deskripsi	Contoh	Hasil
and	Kedua kondisi benar	True and False	False
or	Salah satu kondisi benar	True or False	True
not	Membalik nilai	not True	False

```
x = True
y = False
print(x and y)  # False
print(x or y)   # True
print(not x)    # False
```

4. Operator Penugasan

Digunakan untuk memberikan nilai ke variabel.

Operator	Contoh	Ekivalen dengan
=	a = 5	a = 5
+=	a +=3	a= a + 3
-=	a -= 3	a = a - 3
*=	a *= 3	a = a * 3
/=	a /= 3	a = a * 3
%=	a %= 3	a = a % 3
//=	a //= 3	a = a // 3
**=	a ** = 3	a = a ** 3

```
a = 5
a += 3  # a = 8
a -= 2  # a = 6
a *= 4  # a = 24
a /= 2  # a = 12.0
print(a)
```

5. Operator Bitwise

Digunakan untuk operasi logika biner pada level bit.

Operator	Deskripsi	Contoh	Hasil
&	AND	5 & 3	1
|	OR	5 | 3	7
^	XOR	5 ^ 3	6
~	NOT	~5	-6
<<	Shift Left	5 << 1	10
>>	Shift Right	5 >> 1	2

```
a = 5       # 0101 in binary
b = 3       # 0011 in binary
print(a & b) # 0001 = 1
```

```
print(a | b) # 0111 = 7
print(a ^ b) # 0110 = 6
print(~a)    # 1010 = -6 (in two's complement representation)
print(a << 1) # 1010 = 10
print(a >> 1) # 0010 = 2
```

6. **Operator Keanggotaan**

Digunakan untuk memeriksa apakah suatu nilai ada dalam sebuah koleksi (seperti list, tuple, string, dll).

Operator	Deskripsi	Contoh	Hasil
in	Ada dalam koleksi	‘a’ in ‘apple’	True
not in	Tidak ada dalam koleksi	'b' not in 'apple'	True

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)   # True
print("grape" not in fruits) # True
```

7. **Operator Identitas**

Digunakan untuk memeriksa apakah dua objek memiliki referensi yang sama di memori.

Operator	Deskripsi	Contoh	Hasil
is	Objek yang sama	a is b	True atau False
is not	Objek yang berbeda	a is not b	True atau False

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)   # True, karena a dan b menunjuk ke objek yang sama
print(a is c)   # False, karena a dan c adalah objek yang berbeda meskipun nilainya sama
print(a == c)   # True, karena nilai dari a dan c sama
```

Operator-operator ini adalah dasar dalam Python yang memungkinkan kita melakukan berbagai macam operasi dasar hingga kompleks dalam pemrograman.

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

5

Data Types

Python memiliki berbagai tipe data bawaan yang memudahkan pemrograman. Berikut ini adalah beberapa tipe data dasar di Python:

1. **Tipe Data Numerik**
   - **Integer (int)**: Tipe data untuk bilangan bulat positif atau negatif.

     ```
     x = 10
     y = -5
     ```

   - **Float (float)**: Tipe data untuk bilangan desimal atau bilangan real.

     ```
     pi = 3.14
     g = -9.81
     ```

   - **Complex (complex)**: Tipe data untuk bilangan kompleks, yang memiliki bagian real dan imajiner.

     ```
     z = 2 + 3j
     ```

2. **Tipe Data String (str)**

   Tipe data untuk menyimpan teks atau karakter. String dideklarasikan dengan tanda kutip tunggal (' ') atau kutip ganda (" ").

   ```
   name = "Alice"
   greeting = 'Hello, World!'
   ```

   String dapat diakses per karakter, digabungkan, dan memiliki banyak metode bawaan seperti upper(), lower(), dan replace().

   ```
   print(name.upper())  # ALICE
   ```

3. **Tipe Data Boolean (bool)**

   Tipe data untuk nilai logika True atau False.

   ```
   is_active = True
   ```

```
is_open = False
```

Boolean sering digunakan dalam operasi perbandingan dan logika.

4. Tipe Data Koleksi

a. List

List adalah tipe data yang dapat menampung kumpulan nilai dan bersifat terurut serta dapat diubah. List dideklarasikan dengan tanda kurung siku [ ].

```
fruits = ["apple", "banana", "cherry"]
```

Anda dapat mengakses, menambahkan, menghapus, atau memodifikasi elemen dalam list.

```
fruits.append("orange")
fruits[1] = "grape"
```

b. Tuple

Tuple adalah tipe data terurut yang tidak dapat diubah. Dideklarasikan dengan tanda kurung ( ).

```
coordinates = (10, 20)
```

Karena bersifat tidak dapat diubah, tuple digunakan untuk menyimpan data yang tidak ingin diubah.

c. Set

Set adalah tipe data koleksi yang tidak terurut dan tidak memiliki elemen yang sama (unik). Dideklarasikan dengan tanda kurung kurawal { }.

```
unique_numbers = {1, 2, 3, 4}
```

Set digunakan untuk operasi himpunan seperti union, intersection, dan difference.

d. Dictionary

Dictionary adalah tipe data yang menyimpan pasangan kunci-nilai (key-value pairs) dan dideklarasikan dengan { }.

```
student = {"name": "Alice", "age": 20}
```

Nilai dalam dictionary dapat diakses menggunakan kunci.

```
print(student["name"])  # Alice
```

5. **Tipe Data Khusus**

   a. **None**

   None adalah tipe data khusus yang merepresentasikan ketidakberadaan nilai atau objek kosong.

```
x = None
```

   b. **Tipe Data Range**

   range adalah tipe data khusus yang merepresentasikan rentang angka, biasanya digunakan dalam perulangan.

```
numbers = range(5)  # Membuat range dari 0 hingga 4
```

6. **Tipe Data Biner**

   a. **Bytes**

   bytes digunakan untuk menyimpan data dalam bentuk biner.

```
b = b"hello"
```

   b. **Bytearray**

   bytearray mirip dengan bytes, tetapi bersifat mutable (dapat diubah).

```
ba = bytearray(5)  # Membuat array byte dengan panjang 5
```

7. **Tipe Data Koleksi Lainnya**

   a. **Frozen Set**

   Frozen Set adalah versi immutable dari set. Setelah didefinisikan, elemen dalam frozen set tidak dapat diubah.

```
fs = frozenset([1, 2, 3])
```

   Contoh Penggunaan Tipe Data

```
# Numerik
x = 42              # int
pi = 3.14159        # float
z = 2 + 3j          # complex
```

```
# String
text = "Hello, Python!"

# Boolean
is_valid = True

# Koleksi
my_list = [1, 2, 3]                # list
my_tuple = (1, 2, 3)               # tuple
my_set = {1, 2, 3}                 # set
my_dict = {"name": "Alice", "age": 25}  # dictionary

# Khusus
nothing = None
```

Memahami tipe data ini penting karena memungkinkan kita untuk memilih tipe yang sesuai untuk data dan kebutuhan operasi tertentu dalam pemrograman Python.

SMK NEGERI SUKASARI

6

User Input

Di Python, **user input** adalah data yang dimasukkan oleh pengguna melalui keyboard. Python menyediakan fungsi bawaan input() untuk menerima input dari pengguna. Berikut adalah penjelasan dan contoh penggunaannya:

1. **Fungsi input()**

   Fungsi input() membaca input dari pengguna sebagai string secara default.

```
name = input("Masukkan nama Anda: ")
print(f'Hello, {name}!")
```

   Output:

```
Masukkan nama Anda: Alice
Hello, Alice!
```

2. **Membaca Input Angka**

   Jika ingin menerima angka (integer atau float), input string harus dikonversi menggunakan fungsi seperti int() atau float().

   a. **Input Integer**

```
age = int(input("Masukkan usia Anda: "))
print(f"Usia Anda adalah {age} tahun.")
```

   Output:

```
Masukkan usia Anda: 25
Usia Anda adalah 25 tahun.
```

   b. **Input Float**

```
height = float(input("Masukkan tinggi badan Anda (cm): "))
print(f"Tinggi Anda adalah {height} cm.")
```

   Output:

```
Masukkan tinggi badan Anda (cm): 175.5
```

```
Tinggi Anda adalah 175.5 cm.
```

3. Input dengan Banyak Nilai (Menggunakan split())

Untuk menerima banyak nilai dalam satu baris, Anda bisa menggunakan metode split().

a. Input List dari Pengguna

```
numbers = input("Masukkan beberapa angka, pisahkan dengan spasi: ").split()
print(f"Angka yang dimasukkan: {numbers}")
```

Output:

```
Masukkan beberapa angka, pisahkan dengan spasi: 1 2 3 4
Angka yang dimasukkan: ['1', '2', '3', '4']
```

b. Konversi ke Tipe Data Lain

```
numbers = list(map(int, input("Masukkan angka, pisahkan dengan spasi: ").split()))
print(f"Angka yang dimasukkan: {numbers}")
```

Output:

```
Masukkan angka, pisahkan dengan spasi: 1 2 3 4
Angka yang dimasukkan: [1, 2, 3, 4]
```

4. Mengatasi Error pada Input (Exception Handling)

Untuk menghindari kesalahan (misalnya pengguna memasukkan data yang salah), gunakan **try-except**.

```
try:
    num = int(input("Masukkan angka: "))
    print(f"Angka Anda: {num}")
except ValueError:
    print("Input harus berupa angka!")
```

Output:

```
Masukkan angka: abc
```

```
Input harus berupa angka!
```

5. **Menggabungkan Input dengan Output**

   Gunakan format string (f-strings atau metode lainnya) untuk menggabungkan input dengan output.

   a. **Menggunakan +**

   ```
   name = input("Masukkan nama Anda: ")
   print("Hello, " + name + "!")
   ```

   b. **Menggunakan f-String**

   ```
   name = input("Masukkan nama Anda: ")
   print(f"Hello, {name}!")
   ```

   c. **Menggunakan format()**

   ```
   name = input("Masukkan nama Anda: ")
   print("Hello, {}!".format(name))
   ```

6. **Input dengan Batasan atau Validasi**

   Anda bisa memvalidasi input agar sesuai dengan kebutuhan.

   a. **Validasi Input Angka Positif**

   ```
   while True:
       try:
           number = int(input("Masukkan angka positif: "))
           if number > 0:
               break
           else:
               print("Angka harus positif!")
       except ValueError:
           print("Input harus berupa angka!")
   print(f"Angka yang dimasukkan: {number}")
   ```

7. **Contoh Lengkap Aplikasi Sederhana**

   a. **Kalkulator Sederhana**

```
print("Kalkulator Sederhana")
try:
    num1 = float(input("Masukkan angka pertama: "))
    num2 = float(input("Masukkan angka kedua: "))
    operator = input("Masukkan operator (+, -, *, /): ")

    if operator == "+":
        result = num1 + num2
    elif operator == "-":
        result = num1 - num2
    elif operator == "*":
        result = num1 * num2
    elif operator == "/":
        result = num1 / num2
    else:
        print("Operator tidak valid!")
        result = None

    if result is not None:
        print(f"Hasil: {result}")
except ValueError:
    print("Input harus berupa angka!")
```

Tips:

- Pastikan input divalidasi untuk menghindari error.
- Gunakan try-except untuk menangkap kesalahan.
- Konversikan input sesuai dengan tipe data yang diperlukan (int, float, dll).

7

Lists & Tuples

Lists & Tuples di Python adalah tipe data koleksi yang digunakan untuk menyimpan beberapa item dalam satu variabel. Meskipun keduanya mirip, terdapat perbedaan penting antara keduanya.

Lists

Ciri-ciri Lists:

- Bersifat **mutable** (dapat diubah setelah dibuat).
- Dideklarasikan menggunakan tanda kurung siku [ ].
- Dapat menyimpan berbagai jenis data (heterogen).

Contoh Penggunaan List

```
# Membuat List
fruits = ["apple", "banana", "cherry"]

# Mengakses Elemen
print(fruits[0])  # Output: apple

# Menambahkan Elemen
fruits.append("orange")
print(fruits)  # Output: ['apple', 'banana', 'cherry', 'orange']

# Mengubah Elemen
fruits[1] = "grape"
print(fruits)  # Output: ['apple', 'grape', 'cherry', 'orange']

# Menghapus Elemen
fruits.remove("cherry")
print(fruits)  # Output: ['apple', 'grape', 'orange']
```

```
# Looping melalui List
for fruit in fruits:
    print(fruit)

# List dengan tipe data campuran
mixed_list = [1, "hello", 3.14, True]
```

Metode Built-in pada List

Metode	Deskripsi	Contoh
append(x)	Menambahkan elemen ke akhir list	fruits.append("mango")
insert(i, x)	Menyisipkan elemen pada indeks tertentu	fruits.insert(1, "kiwi")
remove(x)	Menghapus elemen pertama yang memiliki nilai x	fruits.remove("banana")
pop([i])	Menghapus dan mengembalikan elemen pada indeks i	fruits.pop(1)
sort()	Mengurutkan elemen dalam list	fruits.sort()
reverse()	Membalik urutan elemen dalam list	fruits.reverse()
clear()	Menghapus semua elemen dalam list	fruits.clear()
count(x)	Menghitung jumlah elemen dengan nilai x	fruits.count("apple")
index(x)	Mengembalikan indeks dari elemen pertama dengan nilai x	fruits.index("apple")

Tuples

Ciri-ciri Tuples:

- Bersifat **immutable** (tidak dapat diubah setelah dibuat).
- Dideklarasikan menggunakan tanda kurung biasa ( ).
- Dapat menyimpan berbagai jenis data (heterogen).
- Digunakan untuk data yang tidak ingin diubah.

Contoh Penggunaan Tuple

```
# Membuat Tuple
coordinates = (10, 20, 30)
```

```
# Mengakses Elemen
print(coordinates[0])  # Output: 10

# Tuple dengan satu elemen (harus diakhiri dengan koma)
single_element = (5,)

# Tuple dengan tipe data campuran
mixed_tuple = (1, "hello", 3.14, True)

# Looping melalui Tuple
for coord in coordinates:
    print(coord)
```

Metode Built-in pada Tuple

Metode	Deskripsi	Contoh
count(x)	Menghitung jumlah elemen dengan nilai x	coordinates.count(20)
index(x)	Mengembalikan indeks dari elemen pertama dengan nilai x	coordinates.index(30)

Perbedaan Antara List dan Tuple

Aspek	List	Tuple
Mutable	Ya, dapat diubah	Tidak, immutable
Deklarasi	Menggunakan [ ]	Menggunakan ( )
Kecepatan	Lebih lambat	Lebih cepat
Penggunaan	Untuk data yang sering berubah	Untuk data yang konstan

Konversi Antara List dan Tuple

Anda dapat mengubah list menjadi tuple, dan sebaliknya.

```
# List ke Tuple
fruits = ["apple", "banana", "cherry"]
fruits_tuple = tuple(fruits)
print(fruits_tuple)  # Output: ('apple', 'banana', 'cherry')
```

```
# Tuple ke List
coordinates = (10, 20, 30)
coordinates_list = list(coordinates)
print(coordinates_list)  # Output: [10, 20, 30]
```

Contoh Aplikasi List dan Tuple

a. Menyimpan Daftar Item

```
shopping_list = ["milk", "bread", "eggs"]
```

b. Menyimpan Koordinat

```
location = (12.345, 67.890)  # Tidak akan berubah
```

List lebih fleksibel, sedangkan tuple digunakan untuk memastikan data tetap konsisten.

8

Dicts & Sets

Dictionaries dan Sets di Python adalah tipe data koleksi, tetapi masing-masing memiliki karakteristik yang unik dan berguna dalam berbagai situasi.

Dictionaries

Ciri-ciri Dictionaries:

- Berisi **pasangan key-value** (kunci-nilai).
- Kunci bersifat unik, sementara nilai bisa duplikat.
- Bersifat **mutable** (dapat diubah).
- Dideklarasikan menggunakan { }.

Contoh Penggunaan Dictionary

```
# Membuat Dictionary
student = {
    "name": "Alice",
    "age": 25,
    "major": "Computer Science"
}

# Mengakses Nilai
print(student["name"])  # Output: Alice

# Menambahkan Elemen
student["grade"] = "A"
print(student)  # Output: {'name': 'Alice', 'age': 25, 'major': 'Computer Science', 'grade': 'A'}

# Mengubah Nilai
student["age"] = 26
```

```
print(student)  # Output: {'name': 'Alice', 'age': 26, 'major': 'Computer Science', 'grade': 'A'}

# Menghapus Elemen
del student["major"]
print(student)  # Output: {'name': 'Alice', 'age': 26, 'grade': 'A'}
```

Metode Built-in pada Dictionary

Metode	Deskripsi	Contoh
`keys()`	Mengembalikan semua kunci dalam dictionary	`student.keys()`
`values()`	Mengembalikan semua nilai dalam dictionary	`student.values()`
`items()`	Mengembalikan pasangan key-value sebagai tuple	`student.items()`
`get(key, default)`	Mengakses nilai berdasarkan kunci, dengan nilai default jika tidak ada	`student.get("age", 0)`
`pop(key)`	Menghapus dan mengembalikan nilai berdasarkan kunci	`student.pop("grade")`
`update(dict2)`	Menambahkan atau mengupdate elemen dari dictionary lain	`student.update({"age": 30})`
`clear()`	Menghapus semua elemen dari dictionary	`student.clear()`

Looping pada Dictionary

```
for key, value in student.items():
    print(f"{key}: {value}")
```

Sets

Ciri-ciri Sets:

- Koleksi **elemen unik** (tidak ada duplikasi).

- Bersifat **tidak terurut** dan **tidak diindeks**.
- Bersifat **mutable** (dapat diubah), tetapi elemen di dalamnya harus **immutable**.
- Dideklarasikan menggunakan { } atau fungsi set().

Contoh Penggunaan Set

```
# Membuat Set
fruits = {"apple", "banana", "cherry"}

# Menambahkan Elemen
fruits.add("orange")
print(fruits)  # Output: {'orange', 'banana', 'apple', 'cherry'}

# Menghapus Elemen
fruits.remove("banana")
print(fruits)  # Output: {'orange', 'apple', 'cherry'}

# Mendeteksi Elemen
print("apple" in fruits)  # Output: True

# Membuat Set dari List
numbers = set([1, 2, 2, 3, 4])
print(numbers)  # Output: {1, 2, 3, 4}
```

Operasi pada Set

Operasi	Deskripsi	Contoh
`union()`	Menggabungkan dua set	`set1.union(set2)`
`intersection()`	Mengembalikan elemen yang sama di kedua set	`set1.intersection(set2)`
`difference()`	Elemen di set pertama tapi tidak di set kedua	`set1.difference(set2)`
`issubset()`	Mengembalikan `True` jika set adalah subset dari set lain	`set1.issubset(set2)`
`issuperset()`	Mengembalikan `True` jika set adalah superset dari set lain	`set1.issuperset(set2)`

Looping pada Set

```
for fruit in fruits:
```

```
print(fruit)
```

Frozen Set

Frozen set adalah versi **immutable** dari set.

```
frozen_fruits = frozenset(["apple", "banana", "cherry"])
print(frozen_fruits)
```

Perbedaan Antara Dictionary dan Set

Aspek	Dictionary	Set
Isi	Pasangan key-value	Hanya elemen unik
Mutable	Ya	Ya (kecuali frozen set)
Deklarasi	Menggunakan `{key: value}`	Menggunakan `{value}`
Akses	Elemen diakses melalui kunci	Tidak bisa diakses dengan indeks

Contoh Aplikasi Dictionary dan Set

a. Dictionary untuk Menyimpan Data Pasangan

```
student_scores = {"Alice": 90, "Bob": 85, "Charlie": 92}
```

b. Set untuk Menyimpan Data Unik

```
unique_ids = {101, 102, 103}
```

9

Loops

Loops dalam Python

Loops digunakan untuk menjalankan sekelompok instruksi secara berulang hingga suatu kondisi tertentu terpenuhi. Python mendukung dua jenis loop utama: **for loop** dan **while loop**.

1. for Loop

for loop digunakan untuk iterasi melalui elemen dalam koleksi (seperti list, tuple, dictionary, set, atau string).

Sintaks

```
for item in iterable:
    # blok kode
```

Contoh Penggunaan

a. Iterasi Melalui List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

b. Iterasi dengan Range

```
for i in range(5):
    print(i)
```

Output:

```
0
1
```

```
2
3
4
```

c. Iterasi dengan Index

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

Output:

```
0: apple
1: banana
2: cherry
```

d. Iterasi Melalui String

```
word = "Python"
for char in word:
    print(char)
```

Output:

```
P
y
t
h
o
n
```

2. while Loop

while loop digunakan untuk menjalankan blok kode selama suatu kondisi bernilai True.

Sintaks

```
while condition:
    # blok kode
```

Contoh Penggunaan

a. Loop dengan Kondisi

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Output:

```
0
1
2
3
4
```

b. Loop dengan Input

```
password = ""
while password != "12345":
    password = input("Masukkan password: ")
print("Akses diberikan!")
```

3. Control Statements dalam Loop

a. break

Menghentikan loop sepenuhnya.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```

b. continue

Melewati iterasi saat ini dan melanjutkan ke iterasi berikutnya.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:

```
0
1
3
4
```

c. else dengan Loop

Blok else pada loop akan dijalankan jika loop selesai tanpa terkena break.

```
for i in range(5):
    print(i)
else:
    print("Loop selesai tanpa break!")
```

Output:

```
0
1
2
3
4
Loop selesai tanpa break!
```

4. Nested Loops

Loop di dalam loop disebut **nested loop**.

Contoh

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")
```

Output:

```
i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1
i=2, j=0
i=2, j=1
```

5. **Infinite Loop**

Hati-hati dengan **infinite loop** karena dapat membuat program tidak berhenti.

Contoh Infinite Loop

```
while True:
    print("Ini loop tak terbatas!")
```

Untuk menghentikannya, Anda bisa menekan **Ctrl + C**.

Contoh Aplikasi Loops

a. Mencari Bilangan Prima

```
n = 10
for num in range(2, n + 1):
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        print(num)
```

b. Menampilkan Pola

```
rows = 5
for i in range(1, rows + 1):
    print("*" * i)
```

Output:

```
*
**
***
```

```
****
*****
```

Loops sangat berguna untuk mengotomasi tugas-tugas yang berulang dalam berbagai skenario!

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI